HIT137

Group Assignment 2 (20% Mark)

“assignment2.zip” contains all the files for this assignment.

You are required to create a GitHub repository and add all your group mates to it (make sure to keep it public, not private). You should do this before you start the assignment.

All the answers and contributions should be recorded in GitHub till you submit the assignment.

Submission Guidelines:

- Include your GitHub Repository link in a text file “github_link.txt”.
- Zip all the programming files and outputs and “github_link.txt” and upload them to Learline.

Late submissions: A late submission penalty of 5% of the total available marks per day will apply.

Question 1

Write a program in a file named `cipher.py` that reads the text file `raw_text.txt`, encrypts its contents using the scheme described below, and writes the result to `encrypted_text.txt`. You must then write a function that decrypts that file, and a function that verifies the decryption was successful.

Requirements

The encryption should take two user inputs, `shift1` and `shift2`, which are non-negative integers. Every character in the file is transformed according to the rules below:

- For lowercase letters:
  - If the letter is in the first half of the alphabet (`a–n`): shift forward by `shift1 * shift2` positions
  - If the letter is in the second half (`o–z`): shift backward by `shift1 + shift2` positions
- For uppercase letters:
  - If the letter is in the first half (`A–M`): shift backward by `shift1` positions
  - If the letter is in the second half (`N–Z`): shift forward by `shift2²` positions (shift2 squared)

- For digits (`0–9`):
  - Shift forward by `shift1 - shift2`
- Other characters:
  - Spaces, tabs, newlines, punctuation, symbols remain unchanged

Main Functions to Implement

`encrypt_file(shift1: int, shift2: int, input_path: str, output_path: str) -> None:`

Reads from "raw_text.txt" and writes encrypted content to "encrypted_text.txt".

`decrypt_file(shift1: int, shift2: int, input_path: str, output_path: str) -> None:`

Reads from "encrypted_text.txt" and writes the decrypted content to "decrypted_text.txt".

`verify_files(original_path: str, decrypted_path: str) -> bool:`

Compares "raw_text.txt" with "decrypted_text.txt" and prints whether the decryption was successful or not.

Program Behaviour

When run, your program should automatically:

1. Prompt the user for shift1 and shift2 values
2. Encrypt the contents of "raw_text.txt"
3. Decrypt the encrypted file
4. Verify the decryption matches the original

Question 2

Write a program that reads mathematical expressions from `input.txt` (one per line), evaluates each expression, and writes the results to an output file.

Your solution should be built from plain functions, no classes. You should use recursive descent parsing, where each level of operator precedence is handled by its own function, and parenthesised sub-expressions are evaluated by recursing back to the top of the grammar.

Requirements

- The evaluator must correctly handle the five binary operators `(+, -, *, /, %)`
- Exponentiation `^`
- Parentheses (nested to any depth

- Unary negation (e.g., `-5`, `--5`, `-(3+4)`).
- Unary `+` is not supported and should produce an error.
- Unary negation may appear at the start of an expression, after an opening parenthesis, or after any operator (e.g., `3 * -2` is valid).
- Implicit multiplication is valid expression (two adjacent numbers e.g. `2 3` are not implicit multiplication).

Precedence and associativity

From lowest to highest binding:

Level	Operators	Associativity
1	`+ -`	Left
2	`* / %` and implicit multiplication	Left
3	unary `-`	Prefix
4	`^`	Right

Numbers

A number literal is one or more digits, optionally followed by a single `.` and one or more digits.

Output Format

Your program must produce a single output file called `output.txt` in the same directory as the input file. Each expression produces a four-line block (`Input`, `Tree`, `Tokens`, `Result`), with blocks separated by a blank line.

Input

- The original expression exactly as read from the file

Tree

- Separated by single spaces; or ERROR.
- A number literal is displayed as its formatted value.
- A binary operation is displayed as (`op left right`), the operator symbol comes first, followed by the left and right sub-trees, all separated by single spaces, enclosed in parentheses.
- A unary negation is displayed as (`neg` operand), using the word `neg`.
- Implicit multiplication appears in the tree as `*`.

Tokens

- Each token in the format `[TYPE:value]`, separated by single spaces, ending with `[END]` or `ERROR`
- Valid token types are: `NUM` (a numeric literal), `OP` (one of `+`, `-`, `*`, `/`, `%`, `^`), `LPAREN` (a `(`), `RPAREN` (a `)`), and `END` (end of input).

- Unary negation is not folded into the number token. The expression `-5` produces the tokens `[OP:-] [NUM:5] [END]`, not `[NUM:-5] [END]`.

Result

- The computed value, or `ERROR`
- If the value is a whole number (e.g. `8.0`) display it with no decimal point (e.g. `8`). Otherwise round to 4 decimal places.

Output Formatting

- Input: line shows the original expression exactly as read from the file.
- Tree: line representing the parse tree, or `ERROR`.
- Tokens: line shows each token in the format `[TYPE:value]` separated by spaces, ending with `[END]`, or `ERROR`.
- Result: line shows the computed value, or `ERROR`. If the result is a whole number (e.g., 8.0), display it without the decimal point (e.g., 8). Otherwise rounded to 4 decimal places.

Required Interface

Create a file “evaluator.py” that defines:

```
def evaluate_file(input_path: str) -> list[dict]:
```

- `input_path` is a string to the input text file.
- The function writes `output.txt` to the same directory as the input file.
- The function returns a list of dictionaries, one per expression, for example:

```
[
      {"input": "3 + 5", “tree”: “(+ 3 5)”, "tokens": "
[NUM:3] [OP:+] [NUM:5] ", "result": 8},
      {"input": "3 @ 5", “tree”: “ERROR”, "tokens":
"ERROR", "result": "ERROR"},
 ]
```

The `result` value is a float on success or the string "ERROR" on failure. The “tree” and "tokens" value are strings.

A sample input file (`sample_input.txt`) and the expected output (`sample_output.txt`) are provided.